

 main.py

Visualize

Run



```
3 def word_subsets(words1, words2):
6     for word in words2:
7         count = Counter(word)
8
9         for char in count:
10             required[char] = max(required[char], count[char])
11
12     result = []
13
14     for word in words1:
15         count = Counter(word)
16
17         if all(count[c] >= required[c] for c in required):
18             result.append(word)
19
20     return result
21
22
23 # Input
24 words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
25 words2 = ["e", "o"]
26
27 print("Universal words:", word_subsets(words1, words2))
```

Output

Universal words: ['facebook', 'google', 'leetcode']

=== Code Execution Successful ===

main.py

Visualize

Run

1

def subsets_containing(nums, x):

4

def backtrack(index, path):

11

path.append(nums[index])

12

backtrack(index + 1, path)

13

path.pop()

14

15

Exclude element

16

backtrack(index + 1, path)

17

18

backtrack(0, [])

19

return result

20

21

22

Input

23

E = [2, 3, 4, 5]

24

x = 3

25

26

answer = subsets_containing(E, x)

27

28

print("Subsets containing", x, ":")

29

for subset in answer:

30

print(subset)

Output

Subsets containing 3 :

[2, 3, 4, 5]

[2, 3, 4]

[2, 3, 5]

[2, 3]

[3, 4, 5]

[3, 4]

[3, 5]

[3]

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def generate_subsets(nums):
2     nums.sort()
3     result = []
4
5     def backtrack(index, path):
6         result.append(path[:])
7
8         for i in range(index, len(nums)):
9             path.append(nums[i])
10            backtrack(i + 1, path)
11            path.pop()
12
13    backtrack(0, [])
14    return result
15
16
17 # Input
18 A = [1, 2, 3]
19
20 print("Subsets:")
21 for subset in generate_subsets(A):
22     print(subset)
```

Output

Subsets:
[]
[1]
[1, 2]
[1, 2, 3]
[1, 3]
[2]
[2, 3]
[3]

=== Code Execution Successful ===



main.py

Visualize

Run



```
1 def hamiltonian_cycle(n, edges):
11     def backtrack():
15         for v in graph[path[-1]]:
16             if v not in visited:
17                 if backtrack():
20                     return True
21             path.pop()
22             visited.remove(v)
23         return False
24     return backtrack()
25
26 # Input
27
28
```

Output

Hamiltonian cycle exists:

=== Code Execution Success



main.py

Visualize

Run

```
1 def hamiltonian_cycle(n, edges):
11     def backtrack():
26         return False
27
28     if backtrack():
29         return True, path + [0]
30
31     return False, []
32
33
34 # Input
35 n = 5
36 edges = [
37     (0,1), (1,2), (2,3), (3,0),
38     (0,2), (2,4), (4,0)
39 ]
40
41 exists, cycle = hamiltonian_cycle(n, edges)
42
43 print("Hamiltonian cycle exists:", exists)
44 if exists:
45     print("Cycle:", cycle)
```

Output

Hamiltonian cycle exists: False

=== Code Execution Successful ===



main.py

Visualize

Run

















JS

TS

```
1 def graph_coloring_k(n, edges, k):
16     def solve(v):
20         for c in range(1, k + 1):
21             if safe(v, c):
24                 solve(v + 1)
26
27         color[v] = 0
28
29         return False
30
31     if solve(0):
32         return color
33     return None
34
35
36 # Input
37 n = 4
38 k = 3
39 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
40
41 answer = graph_coloring_k(n, edges, k)
42
43 print("Color assignment:", answer)
```

Output

Color assignment: [1, 2, 3, 2]

=== Code Execution Successful ===

main.py

Visualize

Run

1 def graph_coloring(n, edges):
16 def solve(v):
20 for c in range(1, n + 1):
21 if is_safe(v, c):
24 if solve(v + 1):
26 return True
27 color[v] = -1
28 return False
29 solve(0)
30 return color
31
32 # Input
33 n = 4
34 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
35
36 colors = graph_coloring(n, edges)
37
38 print("Vertex colors:", colors)
39
40 print("Number of colors:", max(colors))

Output

Vertex colors: [1, 2, 3, 2]
Number of colors: 3

=== Code Execution Successful ===



main.py

VisualizeRun

```
1 def unique_permutations(nums):
5     def backtrack(path, used):
10         for i in range(len(nums)):
14             if i < 0 and nums[i] == nums[i-1] and not used[i-1]:
16                 continue
17             used[i] = True
18             path.append(nums[i])
19
20             backtrack(path, used)
21
22             path.pop()
23             used[i] = False
24
25     backtrack([], [False] * len(nums))
26     return result
27
28
29 # Input
30 nums = [1, 1, 2]
31
32 print("Unique permutations:")
33 for p in unique_permutations(nums):
34     print(p)
```

Output

Unique permutations:
[1, 1, 2]
[1, 2, 1]
[2, 1, 1]

=== Code Execution Successful ===



main.py

Visualize

Run

```
1 def permutations(nums):
2     def backtrack(path, used):
3
4         for i in range(len(nums)):
5             if not used[i]:
6                 used[i] = True
7                 path.append(nums[i])
8
9                 backtrack(path, used)
10
11                 path.pop()
12                 used[i] = False
13
14     backtrack([], [False] * len(nums))
15     return result
16
17 # Input
18 nums = [1, 2, 3]
19
20 print("Permutations:")
21 for p in permutations(nums):
22     print(p)
```

Output

Permutations:
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]

=== Code Execution Successful ===



main.py

Visualize

Run

```
1 def combination_sum2(candidates, target):
5     def backtrack(start, target, path):
6
10         for i in range(start, len(candidates)):
11             if i > start and candidates[i] == candidates[i - 1]:
12                 continue
13
14             if candidates[i] > target:
15                 break
16
17             path.append(candidates[i])
18             backtrack(i + 1, target - candidates[i], path)
19             path.pop()
20
21         backtrack(0, target, [])
22         return result
23
24
25 # Input
26 candidates = [10, 1, 2, 7, 6, 1, 5]
27 target = 8
28
29 print("Combinations:", combination_sum2(candidates, target))
```

Output

Combinations: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

=== Code Execution Successful ===



main.py

Visualize

Run

```
1 def combination_sum(candidates, target):
5     def backtrack(start, target, path):
6         if target == 0:
7             result.append(path)
8             return
9
10        for i in range(start, len(candidates)):
11            if candidates[i] > target:
12                break
13
14            path.append(candidates[i])
15            backtrack(i, target - candidates[i], path)
16            path.pop()
17
18    backtrack(0, target, [])
19    return result
20
21
22 # Input
23 candidates = [2, 3, 6, 7]
24 target = 7
25
26 print("Combinations:", combination_sum(candidates, target))
```

Output

Combinations: [[2, 2, 3], [7]]

=== Code Execution Successful ===



main.py

VisualizeRun

```
1 def sum_subarray_mins(arr):
16     stack = []
17
18     for i in range(n - 1, -1, -1):
19         while stack and arr[stack[-1]] >= arr[i]:
20             stack.pop()
21
22         right[i] = stack[-1] - i if stack else n - i
23         stack.append(i)
24
25     result = 0
26
27     for i in range(n):
28         result += arr[i] * left[i] * right[i]
29
30     return result % MOD
31
32
33 # Input
34 arr = [3, 1, 2, 4]
35
36 print("Sum of subarray minimums:", sum_subarray_mins(arr))
```

Output

Sum of subarray minimums: 17

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def find_target_sum_ways(nums, target):
2     dp = {0: 1}
3
4     for num in nums:
5         new_dp = {}
6
7         for total, count in dp.items():
8             new_dp[total + num] = new_dp.get(total + num, 0) + count
9             new_dp[total - num] = new_dp.get(total - num, 0) + count
10
11         dp = new_dp
12
13     return dp.get(target, 0)
14
15
16 # Input
17 nums = [1, 1, 1, 1, 1]
18 target = 3
19
20 print("Number of ways:", find_target_sum_ways(nums, target))
```

Output

Number of ways: 5

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def solve_sudoku(board):
2     solve()
37     return board
38
39
40 # Input
41 board = [
42     ["5","3",".",".",".", "7",".",".","."],
43     ["6",".",".", "1","9","5",".","."],
44     [".","9","8",".",".", "6",".","."],
45     ["8",".",".", "6",".",".", "3"],
46     ["4",".", "8",".", "3",".", "1"],
47     ["7",".", "2",".", "6"],
48     [".","6",".", "2","8"],
49     [".",".", "4","1","9",".", "5"],
50     [".",".", "8",".", "7","9"]
51 ]
52
53 solve_sudoku(board)
54
55 # Output
56 for row in board:
57     print(row)
```

Output

```
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
```

=== Code Execution Successful ===



















JS

TS

main.py

Visualize

Run

```
1 def is_valid_solution(solution):
2     for i in range(len(solution)):
3         for j in range(i + 1, len(solution)):
4             if solution[i] == solution[j]:
5                 return False
6             if abs(solution[i] - solution[j]) == abs(i - j):
7                 return False
8     return True
9
10
11 # Input
12 rows = 8
13 columns = 10
14 solution = [1, 3, 5, 7, 9, 2, 4, 6]
15
16 print("Board:", rows, "x", columns)
17 print("Solution:", solution)
18 print("Valid:", is_valid_solution(solution))
```

Output

Board: 8 x 10
Solution: [1, 3, 5, 7, 9, 2, 4, 6]
Valid: False

=== Code Execution Successful ===

main.py

Visualize

Run

1 def solve_n_queens(n):

15 def backtrack(row):

16 if row == n:

17 solutions.append(''.join(''.join(board[i][j]) for i in range(n)))

18 return

19

20 for col in range(n):

21 if safe(row, col):

22 board[row][col] = "Q"

23 backtrack(row + 1)

24 board[row][col] = "."

25

26 backtrack(0)

27 return solutions

28

29

30 # Input

31 N = 4

32

33 # Output

34 solutions = solve_n_queens(N)

35 for row in solutions[0]:

36 print(row)

Output

.Q..

...Q

Q...

..Q.

=== Code Execution Successful ===